GitHub Issues Roadmap & Milestone Specification

Project: Autonomous Livestream Companion Architecture (ALCA)

Author: Patrick Mannix

Deliverables & Evidence Checklist

Links

- **Repository Link:** https://github.com/patrick-mannix/alca-livestream-companion
- **Project Board Link (Roadmap view):** https://github.com/users/patrick-mannix/projects/1/views/2

Evidence Checklist

- [x] Start Date and Target Date fields created in Table/Roadmap view
- [x] Dates assigned to all active issues and milestones
- [x] Dependencies enabled and configured in Project settings
- [x] All 5 required milestones created and labeled (`milestone`)
- [x] Milestones connected to supporting issues via task lists and reference links

Milestone Definitions & Supporting Work

1. [MILESTONE] LVP Defined

- **Start Date:** Oct 14, 2026
- **Target Date:** Oct 21, 2026
- **Label:** `milestone`

Experience

Viewers experience the AI companion as a distinct digital entity on stream that reacts instantly to public chat messages and acknowledges subscription milestone triggers. Simultaneously, the creator can view status indicators confirming the system's operating state.

System Behavior

- **Input:** Raw chat events and webhook triggers from Twitch/YouTube APIs.
- **Process:** Python WebSocket listener ingests events, sorts them via an async priority queue, and routes them through a basic LangChain prompt formatting layer.
- **Output:** Formatted text response passed to a text-to-speech (TTS) engine and rendered on a browser-based stream overlay.

Scope and Limitations

- **Included:** Real-time chat polling, subscription priority queuing, basic LLM response generation, and text-to-speech browser overlay audio.
- **Excluded:** Complex visual computer vision stream analysis (the AI watching gameplay), multi-month persistent vector memory, and multi-platform cross-streaming sync.

Components

- Python WebSocket Listener (`/src/listener.py`) — *Linked to Issue #1*
- Priority Queue Router (`/src/queue_manager.py`) — *Linked to Issue #2*
- LangChain LLM Pipeline (`/src/agent.py`) — *Linked to Issue #3*

LVP vs Final Version

The LVP establishes core text-and-audio interaction using simulated or live chat inputs with basic priority handling. The final version expands this by adding full creator audio-to-text override channels, long-term vector memory across sessions, and robust toxicity filtering.

2. [MILESTONE] Prototype v0

- **Start Date:** Oct 22, 2026
- **Target Date:** Nov 04, 2026
- **Label:** milestone
- **Dependencies:** Blocks on *[MILESTONE] LVP Defined*
- **System State:**
    1. Functional Python script successfully connecting to a live Twitch/YouTube chat endpoint via WebSockets.
    2. Priority queue successfully prioritizing subscription events over general chat lines.
    3. Basic local LLM text generation producing contextual replies within a 2-second window.
- **Validation (Testable Checks):**
    1. Run connection script and verify chat packets are logged without crashing.
    2. Simulate a subscription webhook and confirm it interrupts standard chat processing in the queue.
    3. Measure end-to-end response generation time to ensure it stays under target latency limits.
- **Supporting Issues:** Issue #1, Issue #2, Issue #3

3. [MILESTONE] Functional System

- **Start Date:** Nov 05, 2026
- **Target Date:** Nov 18, 2026
- **Label:** milestone
- **Dependencies:** Blocks on *[MILESTONE] Prototype v0*
- **System State:**
    1. Text-to-Speech (TTS) integration successfully rendering audio streams from LLM token outputs.
    2. Browser overlay UI rendering text and audio states on a mock streaming layout (OBS compatible).
    3. Basic safety and token filtering operational to drop malformed or flagged inputs.
- **Validation (Testable Checks):**
    1. Verify that generated text successfully triggers audio playback files via the TTS pipeline.
    2. Test OBS browser source integration to ensure transparency and overlay responsiveness.
    3. Send test toxic strings and confirm the safety filter blocks execution.

- **Supporting Issues:** Issue #4, Issue #5

4. [MILESTONE] Pre-Final Integration

- **Start Date:** Nov 19, 2026
- **Target Date:** Dec 02, 2026
- **Label:** milestone
- **Dependencies:** Blocks on *[MILESTONE] Functional System*
- **System State:**
    1. Complete end-to-end pipeline operating under simulated live stream conditions.
    2. Host audio override channel configured for manual creator intervention.
    3. Repository documentation, setup guides, and README instructions fully updated and polished.
- **Validation (Testable Checks):**
    1. Execute a full simulated 10-minute stream test with high-velocity chat input.
    2. Use the host override channel to pause or redirect the AI mid-stream.
    3. Perform a clean repository clone on a secondary machine to verify installation steps work independently.
- **Supporting Issues:** Issue #6, Issue #7

5. [MILESTONE] Final Presentation

- **Start Date:** Dec 03, 2026
- **Target Date:** Dec 10, 2026
- **Label:** milestone
- **Dependencies:** Blocks on *[MILESTONE] Pre-Final Integration*
- **System State:**
    1. Final slide deck compiled with complete "Audience Reads" and "I Speak" scripting notes, plus Q&A sections.
    2. Runnable PoC demo package and recorded demonstration video assets finalized.
    3. Final submission repository archived and tagged.
- **Validation (Testable Checks):**
    1. Rehearse presentation pacing against time constraints.
    2. Verify live or recorded demo video functions smoothly without runtime errors.
    3. Check that all faculty feedback items from proposal evaluations have been explicitly addressed.
- **Supporting Issues:** Issue #8, Issue #9

Active Issues & Roadmap Sequencing (Table View)

Issue ID & Title	Start Date	Target Date	Milestone Connection	Dependencies

#1: WebSocket Chat Listener Setup	Oct 14, 2026	Oct 21, 2026	[MILESTONE] LVP Defined	None (Root Issue)
#2: Priority Queue Implementation	Oct 22, 2026	Oct 28, 2026	[MILESTONE] Prototype v0	Depends on #1
#3: LangChain Prompt Pipeline	Oct 29, 2026	Nov 04, 2026	[MILESTONE] Prototype v0	Depends on #2
#4: TTS Engine Integration	Nov 05, 2026	Nov 11, 2026	[MILESTONE] Functional System	Depends on #3
#5: Stream Overlay Frontend UI	Nov 12, 2026	Nov 18, 2026	[MILESTONE] Functional System	Depends on #4
#6: Host Audio Override Channel	Nov 19, 2026	Nov 25, 2026	[MILESTONE] Pre-Final Integration	Depends on #5
#7: Repository & README Polish	Nov 26, 2026	Dec 02, 2026	[MILESTONE] Pre-Final Integration	Depends on #6

#8: Slide Deck & Script Finalization	Dec 03, 2026	Dec 07, 2026	[MILESTONE] Final Presentation	Depends on #7
#9: Demo Recording & Packaging	Dec 08, 2026	Dec 10, 2026	[MILESTONE] Final Presentation	Depends on #8

Reflection on Timeline Risk and Sequencing Logic

The foundational issue that must be completed for the ALCA system to function is **Issue #1: WebSocket Chat Listener Setup**. Without establishing a stable, real-time connection to ingest chat events from streaming platforms, no downstream data processing can occur.

The dependency that most strongly shapes the timeline is the linear bridge between **Issue #1 (Chat Ingestion)**, **Issue #2 (Priority Queue Routing)**, and **Issue #3 (LangChain Prompt Pipeline)**. Because chat data must first be received, then filtered/prioritized, and finally formatted before reaching any generative model, these three tasks form a strict sequential dependency chain where each step blocks the next.

If the first critical issue (the WebSocket listener) is delayed or encounters API rate-limiting blocks, a cascading failure chain occurs throughout the project schedule. Specifically, the priority queue cannot be tested with live data, the LangChain routing layer remains unverified, and downstream audio/visual integration (Issues #4 and #5) is pushed back. This would compress the pre-final integration window, threatening the stability of the Proof of Concept demo required for the final presentation. Proper sequencing ensures early technical validation mitigates this risk before user-facing features are built.